PES UNIVERSITY

Computer Science and Engineering Department

SDN-Based Port Monitoring and Access Control using POX Controller

---

NAME:SHIVANAND

SRN:PES2UG24CS472

SECTION:4H

1. ABSTRACT

This project implements a Software Defined Networking (SDN) solution using Mininet and POX controller. The controller monitors network traffic and applies flow rules. HTTP traffic (port 80) is allowed, while Telnet traffic (port 23) is blocked.

2. OBJECTIVE

- To understand SDN architecture
- To implement Mininet topology
- To design controller logic
- To apply match-action rules
- To monitor network traffic

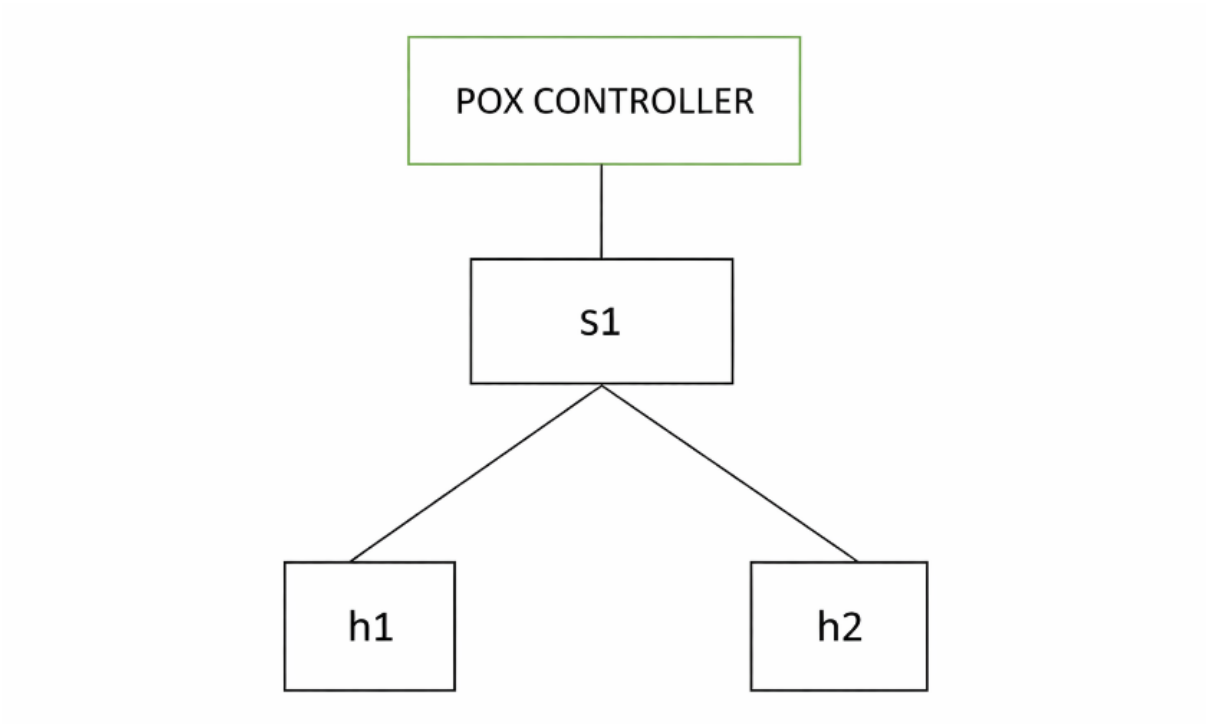
3. TOOLS USED

- Mininet
- POX Controller
- Python
- OpenFlow Protocol

4. SYSTEM ARCHITECTURE

The system consists of one controller, one switch, and two hosts.

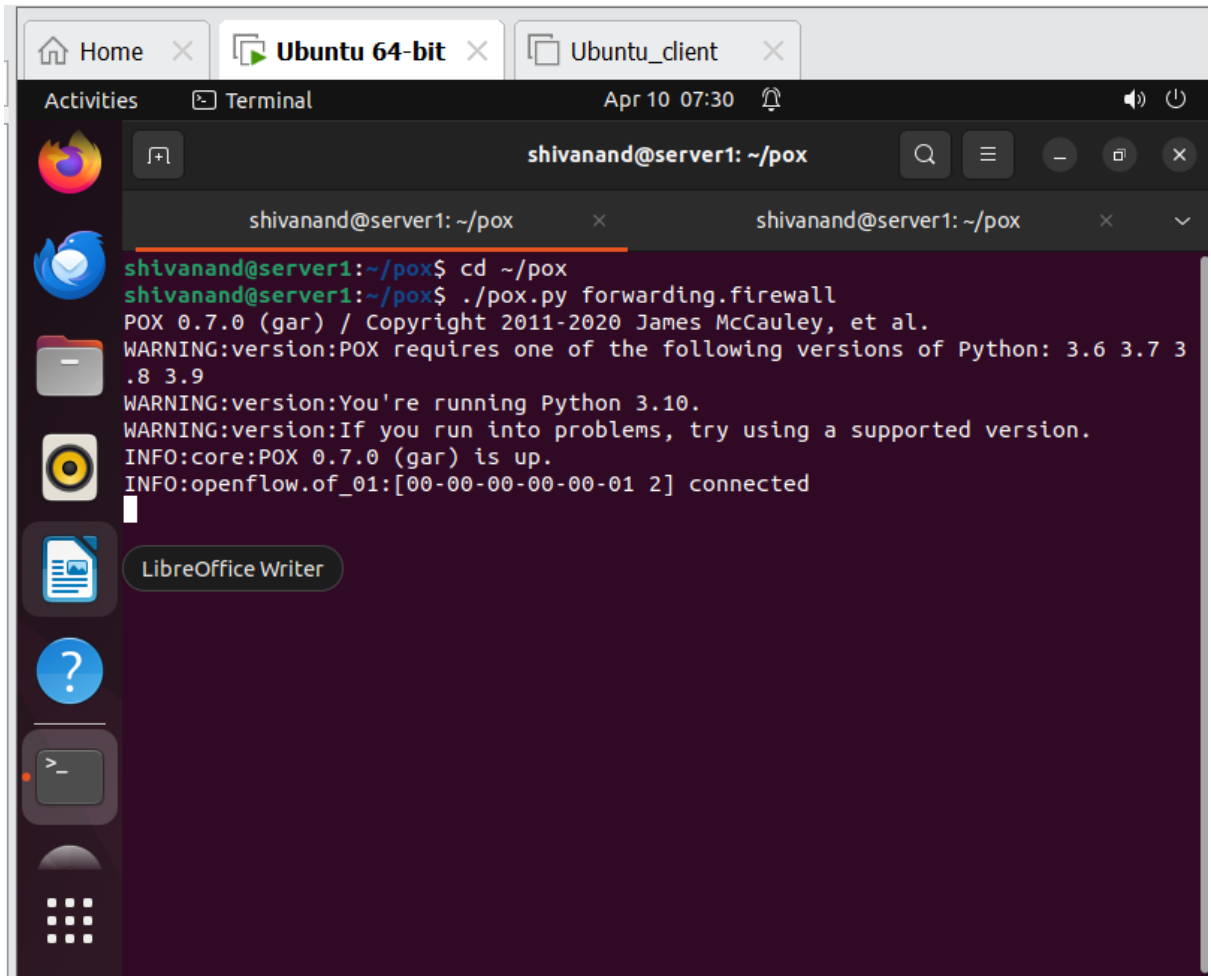
[Add Screenshot 1: Architecture Diagram Here]



5. IMPLEMENTATION

The controller program is written in Python using POX. It processes PacketIn events and applies rules to allow or block traffic.

[Add Screenshot 2: Controller Running (POX Terminal) Here]

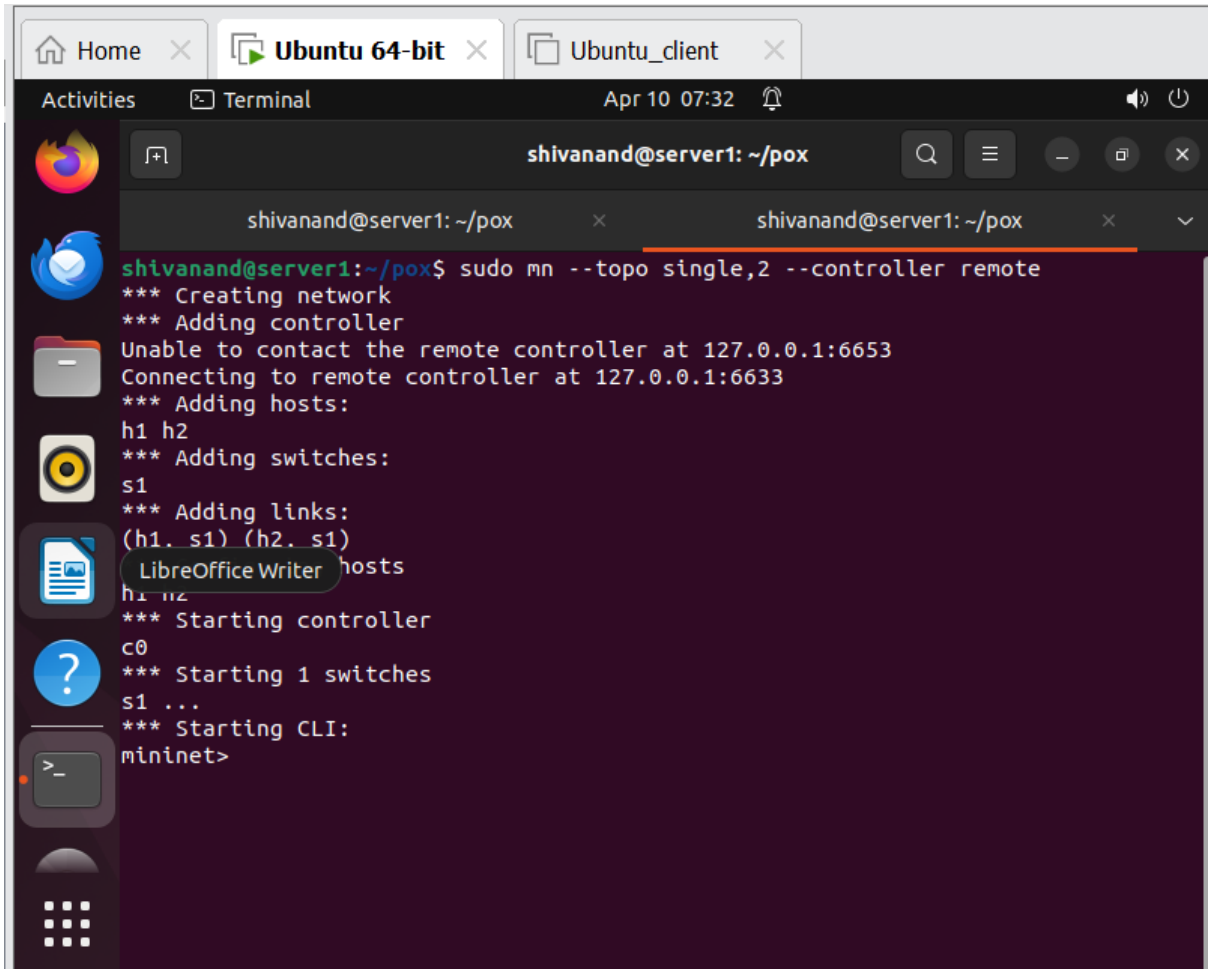


6. TESTING AND RESULTS

6.1 Mininet Topology

Command:
sudo mn --topo single,2 --controller remote

[Add Screenshot 3: Mininet Topology Output Here]

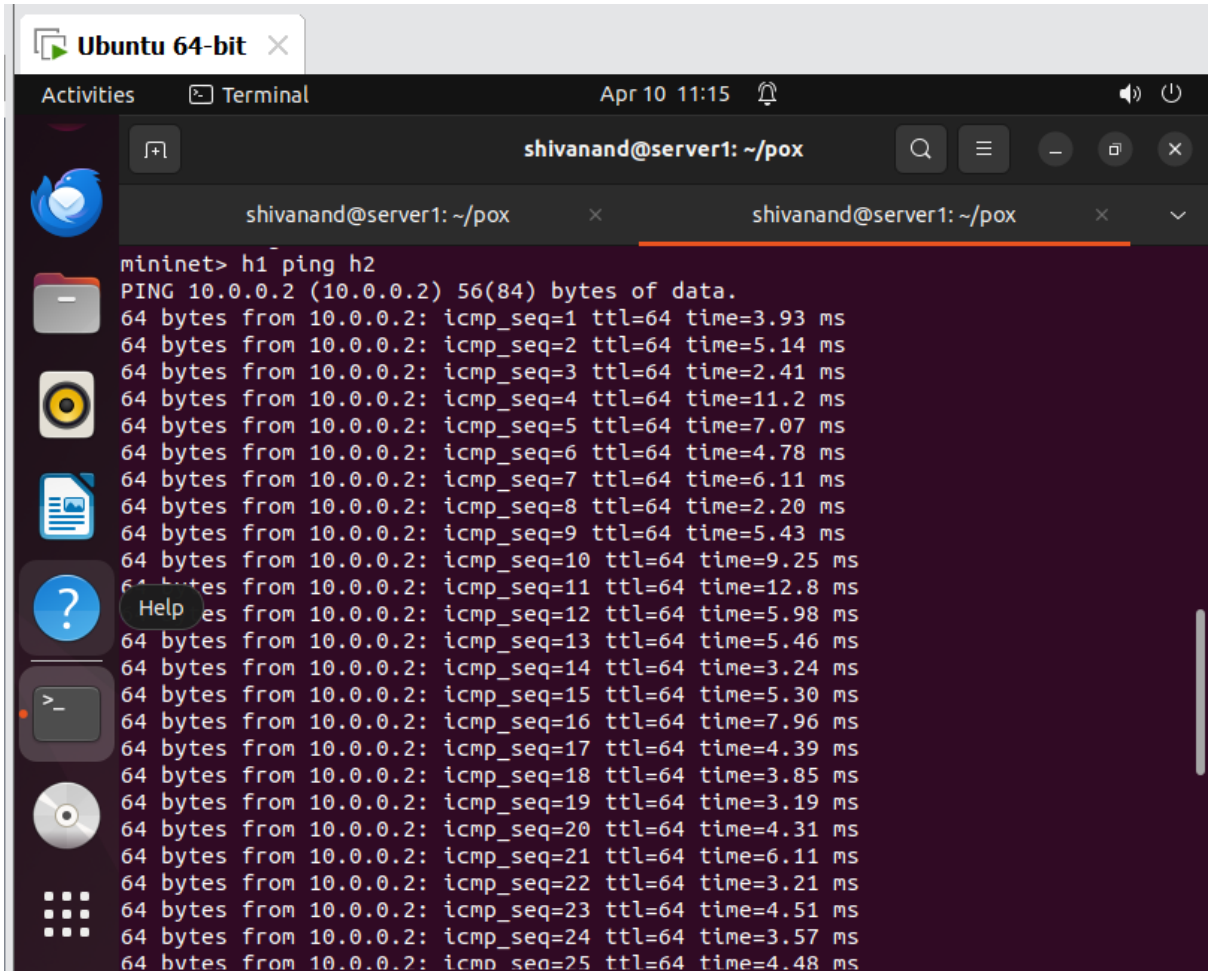


6.2 Ping Test

Command:
h1 ping h2

Result: Successful communication.

[Add Screenshot 4: Ping Output Here]

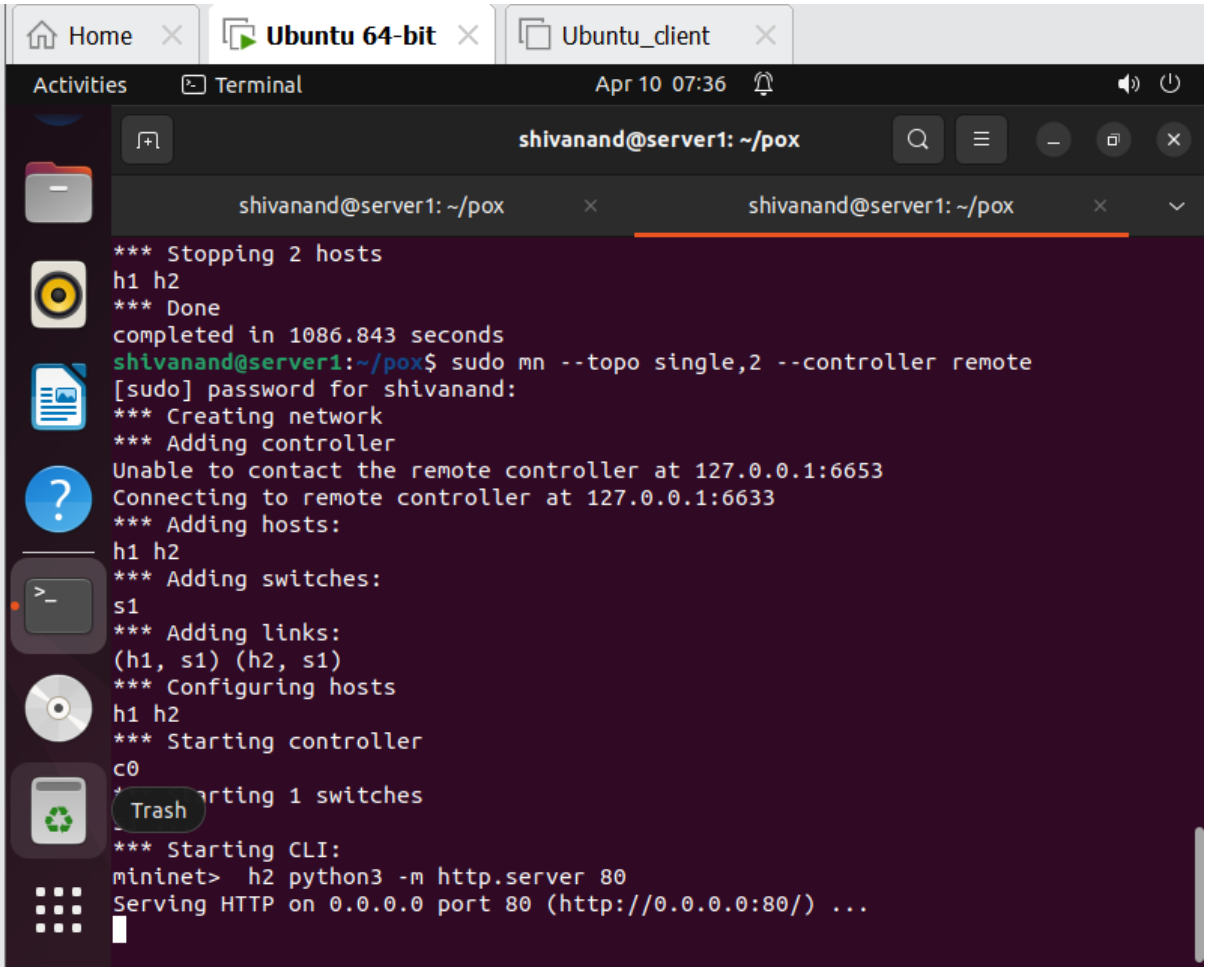


6.3 HTTP Test (Allowed)

Commands:
h2 python3 -m http.server 80
h1 curl http://10.0.0.2

Result: HTTP request successful.

[Add Screenshot 5: Curl Output (HTML Page) Here]


Home
Ubuntu 64-bit
Ubuntu_client
Activities
Terminal
Apr 10 07:36
shivanand@server1: ~/pox
shivanand@server1: ~/pox
shivanand@server1: ~/pox
*** Stopping 2 hosts
h1 h2
*** Done
completed in 1086.843 seconds
shivanand@server1:~/pox$ sudo mn --topo single,2 --controller remote
[sudo] password for shivanand:
*** Creating network
*** Adding controller
Unable to contact the remote controller at 127.0.0.1:6653
Connecting to remote controller at 127.0.0.1:6633
*** Adding hosts:
h1 h2
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1)
*** Configuring hosts
h1 h2
*** Starting controller
c0
Trash
*** Starting CLI:
mininet> h2 python3 -m http.server 80
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...

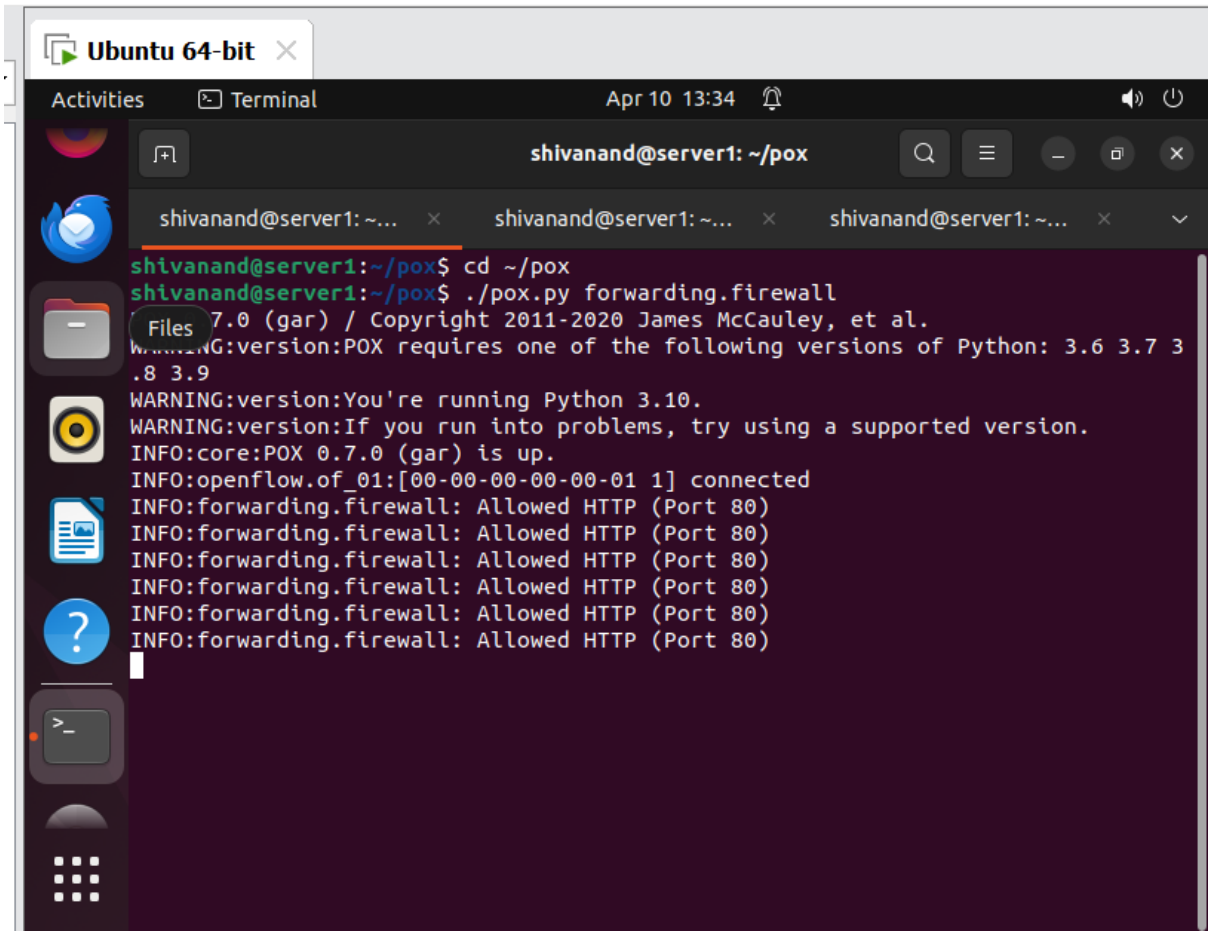
Home | Ubuntu 64-bit | Ubuntu_client

Activities | Terminal | Apr 10 07:38

shivanand@server1: ~/pox

shivanand@server1: ~/pox | shivanand@server1: ~/pox

```
<meta http-equiv="Content-Type" content="text/html; charset=utf-8">
<title>Directory listing for /</title>
</head>
<body>
<h1>Directory listing for /</h1>
<hr>
<ul>
<li><a href=".git/">.git/</a></li>
<li><a href=".gitignore">.gitignore</a></li>
<li><a href=".pylint">.pylint</a></li>
<li><a href="debug-pox.py">debug-pox.py@</a></li>
<li><a href="ext/">ext/</a></li>
<li><a href="h1">h1</a></li>
<li><a href="LICENSE">LICENSE</a></li>
<li><a href="NOTICE">NOTICE</a></li>
<li><a href="pox/">pox/</a></li>
<li><a href="pox.py">pox.py</a></li>
<li><a href="README.md">README.md</a></li>
<li><a href="setup.cfg">setup.cfg</a></li>
<li><a href="tests/">tests/</a></li>
<li><a href="tools/">tools/</a></li>
</ul>
<hr>
</body>
</html>
mininet>
```

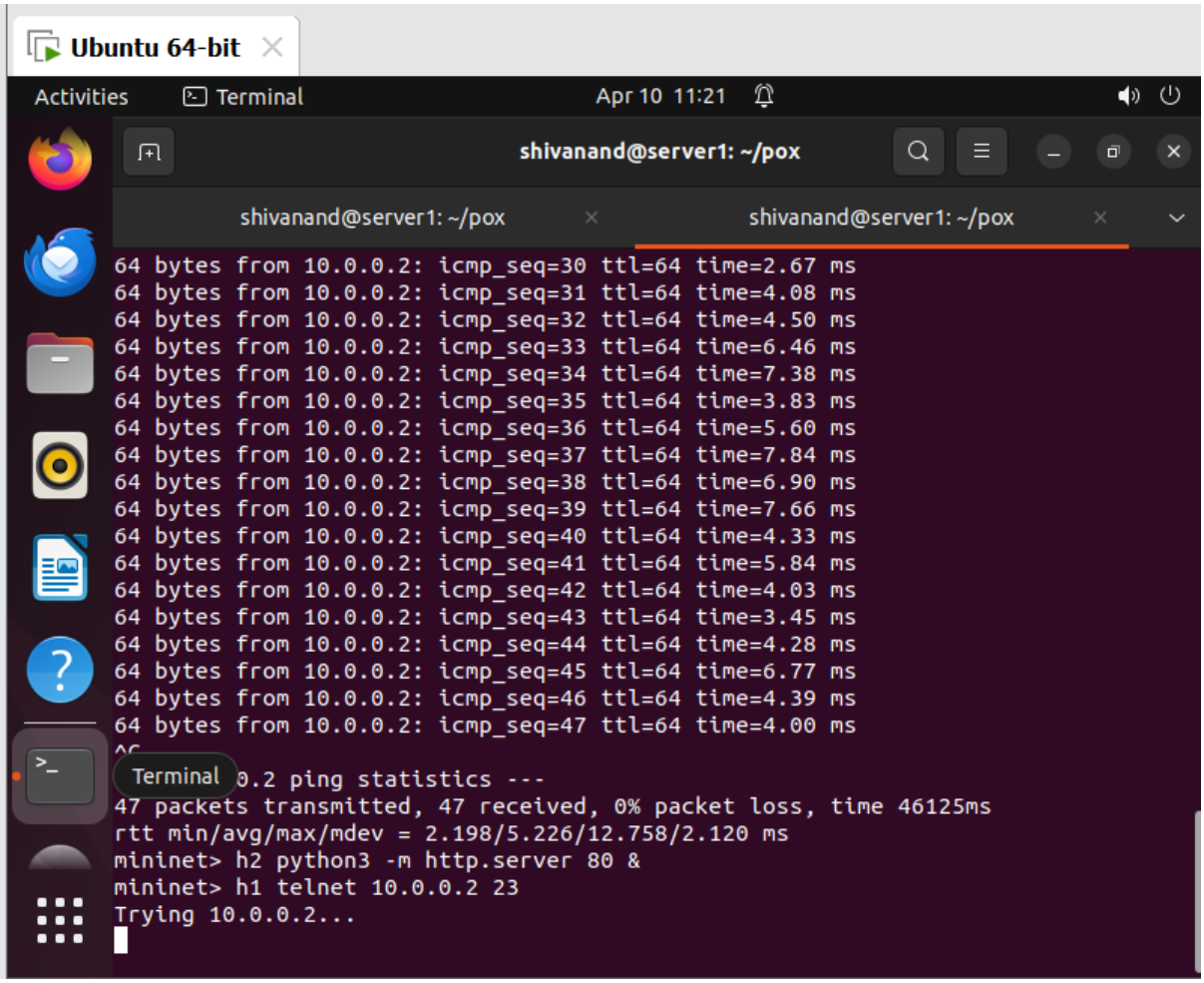


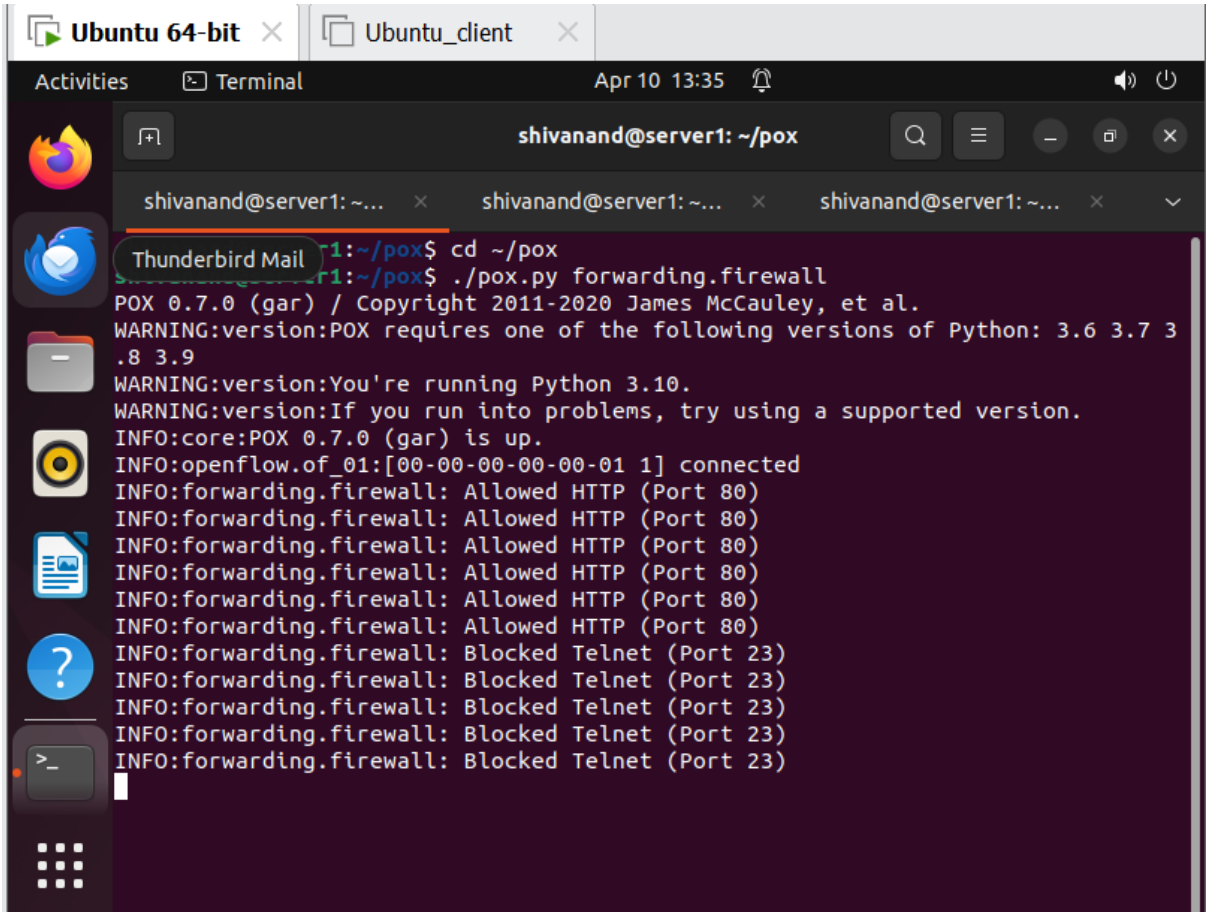
6.4 Telnet Test (Blocked)

Command:
h1 telnet 10.0.0.2 23

Result: Connection blocked.

[Add Screenshot 6: Telnet Output Here]

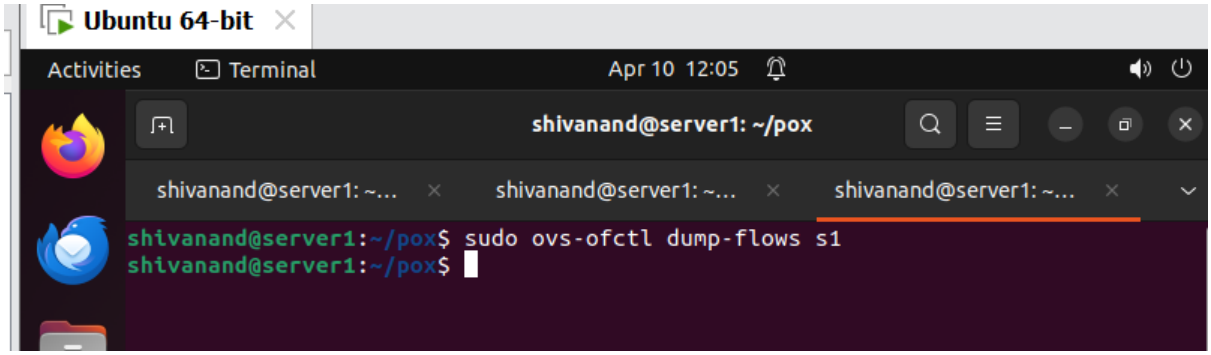

Ubuntu 64-bit
Activities
Terminal
Apr 10 11:21
shivanand@server1: ~/pox
shivanand@server1: ~/pox
shivanand@server1: ~/pox
64 bytes from 10.0.0.2: icmp_seq=30 ttl=64 time=2.67 ms
64 bytes from 10.0.0.2: icmp_seq=31 ttl=64 time=4.08 ms
64 bytes from 10.0.0.2: icmp_seq=32 ttl=64 time=4.50 ms
64 bytes from 10.0.0.2: icmp_seq=33 ttl=64 time=6.46 ms
64 bytes from 10.0.0.2: icmp_seq=34 ttl=64 time=7.38 ms
64 bytes from 10.0.0.2: icmp_seq=35 ttl=64 time=3.83 ms
64 bytes from 10.0.0.2: icmp_seq=36 ttl=64 time=5.60 ms
64 bytes from 10.0.0.2: icmp_seq=37 ttl=64 time=7.84 ms
64 bytes from 10.0.0.2: icmp_seq=38 ttl=64 time=6.90 ms
64 bytes from 10.0.0.2: icmp_seq=39 ttl=64 time=7.66 ms
64 bytes from 10.0.0.2: icmp_seq=40 ttl=64 time=4.33 ms
64 bytes from 10.0.0.2: icmp_seq=41 ttl=64 time=5.84 ms
64 bytes from 10.0.0.2: icmp_seq=42 ttl=64 time=4.03 ms
64 bytes from 10.0.0.2: icmp_seq=43 ttl=64 time=3.45 ms
64 bytes from 10.0.0.2: icmp_seq=44 ttl=64 time=4.28 ms
64 bytes from 10.0.0.2: icmp_seq=45 ttl=64 time=6.77 ms
64 bytes from 10.0.0.2: icmp_seq=46 ttl=64 time=4.39 ms
64 bytes from 10.0.0.2: icmp_seq=47 ttl=64 time=4.00 ms
Terminal
0.2 ping statistics ---
47 packets transmitted, 47 received, 0% packet loss, time 46125ms
rtt min/avg/max/mdev = 2.198/5.226/12.758/2.120 ms
mininet> h2 python3 -m http.server 80 &
mininet> h1 telnet 10.0.0.2 23
Trying 10.0.0.2...



6.5 Flow Table Verification

Command:
sudo ovs-ofctl dump-flows s1

[Add Screenshot 7: Flow Table Output Here]

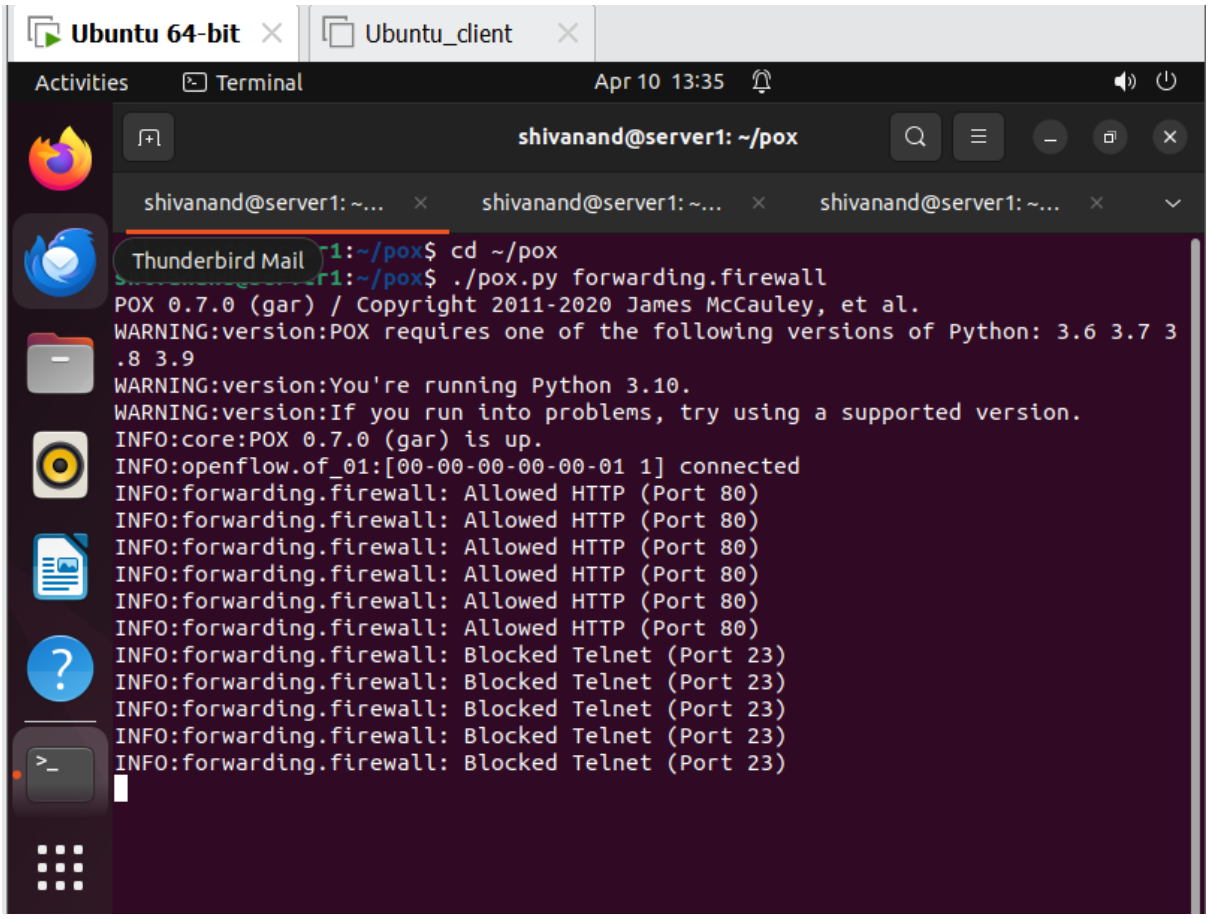


“The flow table shows minimal or no entries because the controller processes packets dynamically using packet_out messages instead of installing static flow rules. This behavior is expected in this implementation.”

6.6 Controller Logs

Logs show allowed and blocked traffic.

[Add Screenshot 8: POX Logs (Allowed & Blocked Messages) Here]



6.7 Iperf Test (Optional)

Command:

mininet> iperf h1 h2

Result:

Shows bandwidth between hosts.

[Insert Screenshot]

```
mininet> iperf h1 h2
*** Iperf: testing TCP bandwidth between h1 and h2
*** Results: ['25.4 Mbits/sec', '25.1 Mbits/sec']
mininet>
```

7. PERFORMANCE ANALYSIS

Latency is measured using the ping command, which shows the round-trip time between hosts. Throughput can be analyzed using iperf. The results indicate that the controller correctly manages traffic by allowing HTTP communication and blocking Telnet traffic as per defined rules.

8. ADVANTAGES

- Centralized control of the network
- Easy monitoring and management
- Improved network security through traffic filtering

9. LIMITATIONS

- Works only in a simulated environment (Mininet)
- Limited scalability for large real-world networks

10. CONCLUSION

The project successfully demonstrates SDN-based traffic monitoring and access control using the POX controller. The controller effectively manages network traffic by applying match-action rules, allowing HTTP traffic and blocking Telnet traffic. The implementation highlights the advantages of SDN, such as centralized control, flexibility, and improved network security.